

# Brainrot Report: Android Architecture 2026

A zero-cap migration brief — Jetpack Compose + Material Design 3. Slang on top, gospel underneath.

## TL;DR (no cap)

The old View system is brainrot: XML inflation, findViewById, brittle state. Modern Android is unidirectional data flow + Compose: state down, events up, smart recomposition only redraws what changed. Material You gives you dynamic color, ripple, and shapes for free.

## Why the View System is Cooked

- XML layout inflation and findViewById hell — runtime cost, no compile-time safety.
- State scattered across Views, Fragments, and adapters — easy to leak, hard to test.
- Fragment lifecycle footguns and memory leaks via retained View references.
- Imperative UI updates drift out of sync with your actual state.

## The Modern Stack

### Unidirectional Data Flow (UDF)

ViewModel exposes immutable UI state via StateFlow. The UI observes it and sends events back up. One source of truth, predictable, testable.

```
data class ScreenUiState(  
    val isLoading: Boolean = false,  
    val items: List<Item> = emptyList(),  
    val error: String? = null,  
)  
  
class FeedViewModel(repo: FeedRepository) : ViewModel() {  
    private val _state = MutableStateFlow(ScreenUiState())  
    val state: StateFlow<ScreenUiState> = _state.asStateFlow()  
    fun onRefresh() { /* update _state.value */ }  
}
```

### Compose Collects State Lifecycle-Aware

```
@Composable  
fun FeedRoute(viewModel: FeedViewModel = hiltViewModel()) {  
    val state by viewModel.state.collectAsStateWithLifecycle()  
}
```

```
FeedScreen(state = state, onRefresh = viewModel::onRefresh)
}
```

## Material 3 Theming

Wrap your app in `MaterialTheme` with `material3`. Opt into dynamic color on Android 12+ so the palette follows the user's wallpaper; fall back to a brand color scheme otherwise.

```
@Composable
fun AppTheme(content: @Composable () -> Unit) {
    val ctx = LocalContext.current
    val colors = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S)
        dynamicLightColorScheme(ctx) else lightColorScheme()
    MaterialTheme(colorScheme = colors, content = content)
}
```

## Migration Strategy (incremental, no big bang)

- Build all new screens and features 100% in Compose from day one.
- Bridge hybrid screens: `ComposeView` inside XML, or `AndroidView` to host legacy Views inside Compose.
- Refactor one screen at a time — swap `setContentView` for `setContent {}`.
- Replace Data Binding / XML with `@Composable` functions; use `remember`, `LaunchedEffect`, and `produceState` for side effects and derived state.
- Adopt Navigation Compose, Hilt for DI, and Room for persistence as you go.

## Architecture Target

Clean-ish layering: UI (Compose) -> ViewModel (UDF/MVI) -> Domain (use cases) -> Data (repositories, Room, network). DI via Hilt. Navigation via Navigation Compose. Adaptive layouts via `WindowSizeClass`.

## Pro Tips

- Keep composables small and stateless; hoist state to the caller.
- Prefer `collectAsStateWithLifecycle` over `collectAsState` to avoid wasted work.
- Use stable/immutable data classes so smart recomposition can skip work.
- Lean on Modifier chains and `MaterialTheme` tokens instead of hardcoded values.
- Write Compose UI tests with `createComposeRule` from the start.

Generated draft — tweak freely. Want code-heavier sections, a sample Compose project, or a pitch-deck version? Say the word.